Cloud Security - PBL

Unit 2 Assignment

IAM & Least Privilege Enforcement

Student name	_____
Roll no. / Division	_____
Date	_____

Problem statement (from the project brief)

Create a rigorous IAM structure. Create distinct **IAM Groups (Admins, Developers, Auditors)** with attached **custom policies** following the **Principle of Least Privilege**. **MFA must be enforced for the Admin group.** Create a **custom IAM Role** that allows an **EC2 instance to access S3 securely without hard-coded credentials**.

Placeholders used in this document: `123456789012` = your AWS account ID, `cloudsec-pbl-app-data` = your S3 bucket name. Replace them with your own values before running anything.

1. Objective

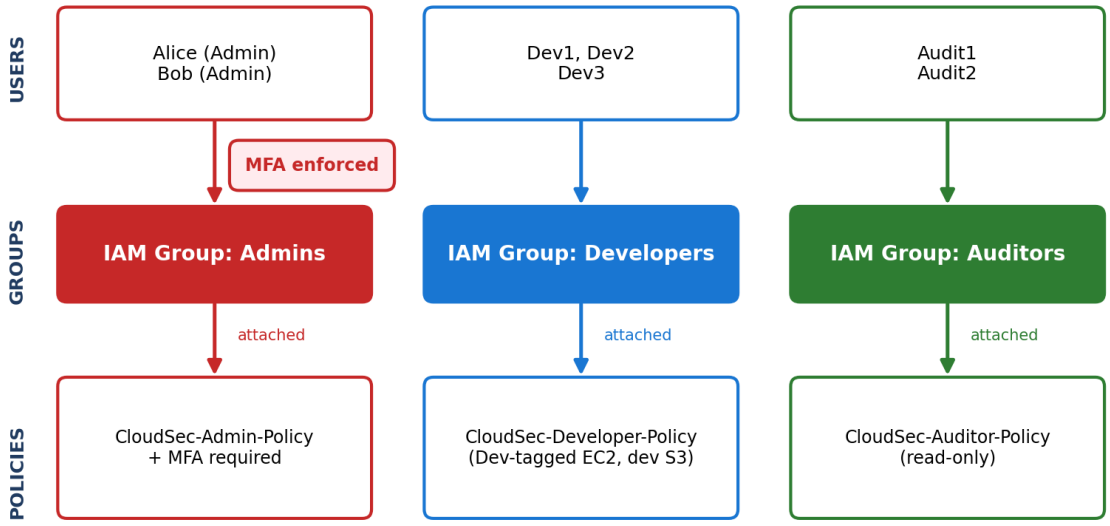
To build an IAM design in which every person and every server has only the permissions it needs, privileged users must use MFA, and an EC2 instance reaches S3 using temporary role credentials instead of stored access keys.

2. Key Concepts

Term	Meaning
IAM User	An identity for one person or application in the AWS account.
IAM Group	A collection of users. Policies attached to the group apply to every member.
IAM Policy	A JSON document that allows or denies actions on resources, optionally with conditions.
Least Privilege	Grant only the minimum permissions required for a task, and nothing more.
MFA	Multi-factor authentication: password plus a one-time code from a device or app.
IAM Role	An identity with permissions that is *assumed* temporarily. It has no password or long-term keys.
Instance Profile	The container that attaches an IAM role to an EC2 instance.

3. Design Overview

Figure 3: IAM Structure - Groups, Policies and MFA



Principle of Least Privilege: each group receives only the permissions its job function needs.

Policy evaluation rule: everything is denied by default; an explicit Allow is needed; an explicit Deny always wins over any Allow. Group policies below use this to keep each role narrow.

4. Permission Matrix

Capability	Admins	Developers	Auditors
Manage IAM users, groups, policies	Yes (with MFA)	No	Read only
Create and delete EC2, RDS, VPC resources	Yes (with MFA)	No	No
Start / stop / reboot EC2	Yes (with MFA)	Only instances tagged Environment=Dev	No
View EC2 and VPC configuration	Yes	Yes	Yes
Read/write S3 objects	Yes	Only dev bucket prefix	No (config only)
View CloudTrail, Config, security settings	Yes	No	Yes
Read CloudWatch logs	Yes	Yes	Yes
Stop or delete CloudTrail	Yes (with MFA)	Explicit Deny	Explicit Deny

5. Step-by-step Implementation

Step 1: Create the groups

```
aws iam create-group --group-name Admins
aws iam create-group --group-name Developers
aws iam create-group --group-name Auditors
```

Console path: IAM > User groups > Create group.

Step 2: Create the custom policies

Create three **customer-managed** policies (IAM > Policies > Create policy > JSON). Do not attach AWS-managed AdministratorAccess to the groups; the custom versions below are narrower.

Policy A: CloudSec-Admin-Policy (admin work is allowed only when MFA was used; before MFA a user can only set up their own MFA)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AdminServicesOnlyWithMFA",
      "Effect": "Allow",
      "Action": ["iam:*","ec2:*","s3:*","rds:*","elasticloadbalancing:*",
                 "autoscaling:*","cloudwatch:*","logs:*","cloudtrail:*",
                 "config:*","kms:*","sts:*"],
      "Resource": "*",
      "Condition": { "Bool": { "aws:MultiFactorAuthPresent": "true" } }
    },
    {
      "Sid": "ManageOwnMFAAndPassword",
      "Effect": "Allow",
      "Action": ["iam:CreateVirtualMFADevice","iam:EnableMFADevice",
                 "iam:ResyncMFADevice","iam:ListMFADevices",
                 "iam:GetUser","iam:ChangePassword"],
      "Resource": ["arn:aws:iam::123456789012:user/${aws:username}",
                   "arn:aws:iam::123456789012:mfa/${aws:username}"]
    },
    {
      "Sid": "ListVirtualMFADevices",
      "Effect": "Allow",
      "Action": "iam:ListVirtualMFADevices",
      "Resource": "*"
    }
  ]
}
```

Why this enforces MFA: the broad Allow has a condition requiring `aws:MultiFactorAuthPresent = true`. A user who signs in with only a password gets no admin permissions; the only things they can do are the self-service MFA actions needed to register a device. After signing in again with an MFA code, the full admin permissions apply.

Policy B: CloudSec-Developer-Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ViewInfrastructure",
      "Effect": "Allow",
      "Action": ["ec2:Describe*","elasticloadbalancing:Describe*",
                 "cloudwatch:Get*","cloudwatch:List*","cloudwatch:Describe*",
                 "logs:Describe*","logs:Get*","logs:FilterLogEvents"],
      "Resource": "*"
    },
    {
      "Sid": "ControlOnlyDevInstances",
      "Effect": "Allow",
      "Action": ["ec2:StartInstances","ec2:StopInstances","ec2:RebootInstances"],
      "Resource": "arn:aws:ec2:*:123456789012:instance/*",
      "Condition": { "StringEquals": { "ec2:ResourceTag/Environment": "Dev" } }
    },
    {
      "Sid": "ListDevBucket",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::cloudsec-pbl-app-data",
      "Condition": { "StringLike": { "s3:prefix": ["dev/*","dev"] } }
    },
    {
      "Sid": "ReadWriteDevObjects",
      "Effect": "Allow",
      "Action": ["s3:GetObject","s3:PutObject"],
      "Resource": "arn:aws:s3:::cloudsec-pbl-app-data/dev/*"
    },
    {
      "Sid": "NeverTouchIAMOrAuditTrail",
      "Effect": "Deny",
      "Action": ["iam:*","organizations:*","cloudtrail:StopLogging",
                 "cloudtrail:DeleteTrail","ec2:TerminateInstances"],
      "Resource": "*"
    }
  ]
}
```

Policy C: CloudSec-Auditor-Policy (read-only visibility for security review; cannot change anything)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadSecurityConfiguration",
      "Effect": "Allow",
      "Action": ["iam:Get*","iam:List*","iam:GenerateCredentialReport",
                 "cloudtrail:Describe*","cloudtrail:Get*","cloudtrail:List*",
                 "cloudtrail:LookupEvents","config:Describe*","config:Get*",
                 "config:List*","ec2:Describe*","rds:Describe*",
                 "logs:Describe*","logs:Get*","logs:FilterLogEvents",
                 "cloudwatch:Describe*","cloudwatch:Get*","cloudwatch:List*"],
      "Resource": "*"
    },
    {
      "Sid": "ReadS3SettingsNotData",
      "Effect": "Allow",
      "Action": ["s3:ListAllMyBuckets","s3:GetBucketPolicy","s3:GetBucketAcl",
                 "s3:GetBucketLogging","s3:GetBucketVersioning",
                 "s3:GetEncryptionConfiguration",
                 "s3:GetBucketPublicAccessBlock"],
      "Resource": "*"
    },
    {
      "Sid": "DenyTrailTampering",
      "Effect": "Deny",
      "Action": ["cloudtrail:StopLogging","cloudtrail:DeleteTrail",
                 "cloudtrail:UpdateTrail","config:Delete*","config:Stop*"],
      "Resource": "*"
    }
  ]
}
```

Step 3: Create policies and attach them to groups

```
aws iam create-policy --policy-name CloudSec-Admin-Policy \
    --policy-document file://admin-policy.json
aws iam create-policy --policy-name CloudSec-Developer-Policy \
    --policy-document file://developer-policy.json
aws iam create-policy --policy-name CloudSec-Auditor-Policy \
    --policy-document file://auditor-policy.json

aws iam attach-group-policy --group-name Admins \
    --policy-arn arn:aws:iam::123456789012:policy/CloudSec-Admin-Policy
aws iam attach-group-policy --group-name Developers \
    --policy-arn arn:aws:iam::123456789012:policy/CloudSec-Developer-Policy
aws iam attach-group-policy --group-name Auditors \
    --policy-arn arn:aws:iam::123456789012:policy/CloudSec-Auditor-Policy
```

Step 4: Create users and add them to groups

```
aws iam create-user --user-name alice
aws iam add-user-to-group --user-name alice --group-name Admins

aws iam create-user --user-name dev1
aws iam add-user-to-group --user-name dev1 --group-name Developers

aws iam create-user --user-name audit1
aws iam add-user-to-group --user-name audit1 --group-name Auditors
```

Give each user console access with a one-time password and require a password change at first login. Do not create access keys for human users.

Step 5: Enforce MFA for the Admin group

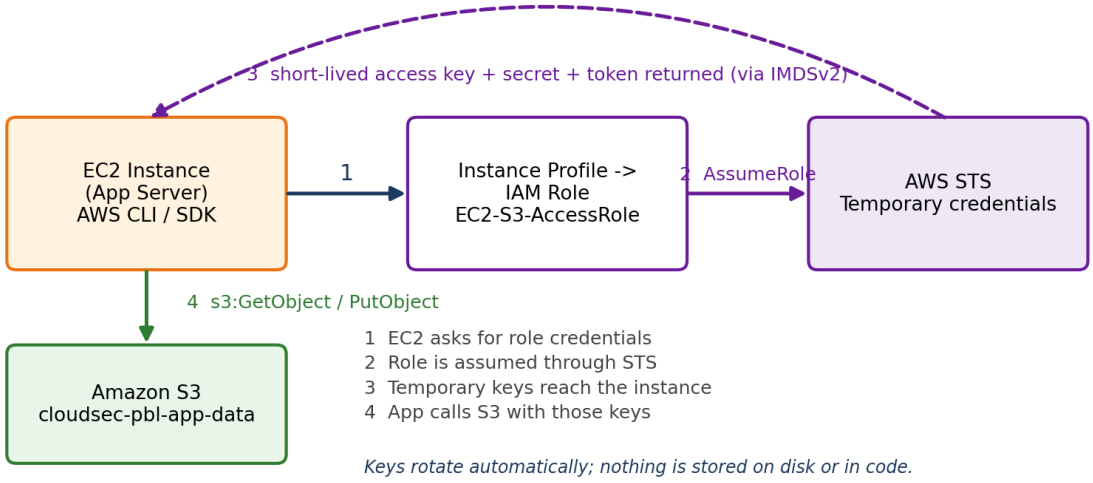
- Sign in as the admin user and open IAM > Users > (user) > Security credentials > Assign MFA device.
- Choose an authenticator app, scan the QR code and enter two consecutive codes.
- Sign out and sign in again with password plus MFA code. Admin permissions now work.

- Verification: a new admin who has not registered MFA can sign in but gets *Access Denied* on EC2, S3 and IAM pages (except MFA setup). This proves enforcement through the policy condition.
- Also enable MFA on the account root user and do not use root for daily work.

6. IAM Role for EC2 to S3 (no hard-coded credentials)

Instead of storing an access key inside the server or code, the instance is given a role. AWS issues short-lived credentials to the instance automatically and rotates them.

Figure 4: EC2 accessing S3 through an IAM Role (no hard-coded credentials)



Step 1: Trust policy (who may assume the role)

Save as `ec2-trust.json`. Only the EC2 service can assume this role.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": { "Service": "ec2.amazonaws.com" },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

Step 2: Permission policy (what the role can do)

Save as `s3-access.json`. It allows only one bucket and one prefix, and only the actions the app needs. It does not allow delete or bucket-level changes.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListAppPrefix",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::cloudsec-pbl-app-data",
      "Condition": { "StringLike": { "s3:prefix": ["app-data/*","app-data"] } }
    },
    {
      "Sid": "ReadWriteAppObjects",
      "Effect": "Allow",
      "Action": ["s3:GetObject","s3:PutObject"],
      "Resource": "arn:aws:s3:::cloudsec-pbl-app-data/app-data/*"
    }
  ]
}
```

Step 3: Create the role, instance profile and attach to EC2

```
aws iam create-role --role-name EC2-S3-AccessRole \
    --assume-role-policy-document file://ec2-trust.json

aws iam create-policy --policy-name EC2-S3-AppDataAccess \
    --policy-document file://s3-access.json

aws iam attach-role-policy --role-name EC2-S3-AccessRole \
    --policy-arn arn:aws:iam::123456789012:policy/EC2-S3-AppDataAccess

aws iam create-instance-profile --instance-profile-name EC2-S3-Profile
aws iam add-role-to-instance-profile \
    --instance-profile-name EC2-S3-Profile --role-name EC2-S3-AccessRole

# attach to an existing instance (or use --iam-instance-profile at launch)
aws ec2 associate-iam-instance-profile --instance-id i-0123456789abcdef0 \
    --iam-instance-profile Name=EC2-S3-Profile
```

The console creates the instance profile automatically when you create an EC2 role (IAM > Roles > Create role > AWS service > EC2). Then attach it with EC2 > Actions > Security > Modify IAM role.

Step 4: Harden metadata access (IMDSv2)

```
aws ec2 modify-instance-metadata-options --instance-id i-0123456789abcdef0 \
    --http-tokens required --http-endpoint enabled
```

Requiring IMDSv2 makes it harder for an attacker who finds a web-app flaw (SSRF) to steal the role's temporary credentials.

7. Testing and Verification

A. Verify the role works from the EC2 instance (no keys configured)

```
# no static keys should exist
ls ~/.aws 2>/dev/null ; env | grep -i AWS_ACCESS

# identity should show an assumed-role, not an IAM user
aws sts get-caller-identity

# allowed actions
echo hello > t.txt
aws s3 cp t.txt s3://cloudsec-pbl-app-data/app-data/t.txt
aws s3 ls s3://cloudsec-pbl-app-data/app-data/

# actions that must be DENIED
aws s3 ls                                   # list all buckets
aws s3 rm s3://cloudsec-pbl-app-data/app-data/t.txt
aws s3 cp t.txt s3://cloudsec-pbl-app-data/other/t.txt
aws iam list-users
```

B. Check that the instance receives temporary credentials (IMDSv2)

```
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
   -H "X-aws-ec2-metadata-token-ttl-seconds: 300")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
   http://169.254.169.254/latest/meta-data/iam/security-credentials/
```

The output shows the role name. Fetching that path adds an AccessKeyId, SecretAccessKey, Token and an *Expiration* time, which proves the credentials are temporary.

C. Expected results

#	Test	Expected result
1	Admin user signs in without MFA	Console loads, but EC2/S3/IAM actions are denied
2	Admin user signs in with MFA	Admin actions succeed
3	Developer starts an EC2 tagged Environment=Dev	Allowed
4	Developer starts an EC2 tagged Environment=Prod	AccessDenied
5	Developer runs iam:ListUsers or terminates an instance	AccessDenied (explicit Deny)
6	Auditor views CloudTrail event history	Allowed
7	Auditor tries to stop an EC2 or edit a policy	AccessDenied
8	EC2 uploads to s3://bucket/app-data/	Allowed
9	EC2 lists all buckets or writes outside app-data/	AccessDenied
10	Search the instance for access keys	None found

You can also test policies without logging in as each user by using the **IAM Policy Simulator** (policysim.aws.amazon.com), or look at the **Access Analyzer / Last accessed** tab to remove unused permissions later.

8. Evidence to Attach (screenshots)

- IAM > User groups showing Admins, Developers, Auditors with their members.
- Each group's Permissions tab showing the attached custom policy.
- JSON of each custom policy.
- Admin user's MFA device listed as assigned, and the failed/passed test with and without MFA.
- Role EC2-S3-AccessRole: Trust relationships and Permissions tabs.

- EC2 instance details showing the IAM role.
- Terminal output of `aws sts get-caller-identity`, the allowed S3 command and the AccessDenied commands.

9. Best Practices Followed

- Permissions given to groups, not to individual users.
- Resource-level limits (specific bucket, prefix, tagged instances) instead of wildcards on everything.
- Explicit Deny for dangerous actions so they cannot be granted by mistake elsewhere.
- MFA required through a policy condition, not just a recommendation.
- No long-term access keys on servers; roles with temporary credentials only.
- IMDSv2 required; CloudTrail protected from being stopped by non-admins.
- Regular review of unused permissions and credential report.

10. Conclusion

The design separates duties among Admins, Developers and Auditors, each limited to the permissions its work needs. MFA is enforced for administrators through the policy itself, so a stolen password alone cannot be used for privileged actions. The EC2 role lets the application use S3 through short-lived, automatically rotated credentials, which removes the risk of leaked access keys in code or on disk. Together these controls apply the Principle of Least Privilege across people and workloads.